

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.
import kagglehub
birdy654_deep_voice_deepfake_voice_recognition_path = kagglehub.dataset_download('birdy654/deep-voice-deepfake-voice-recognition')

print('Data source import complete.')
```

 Data source import complete.

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load
```

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
```

```
# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory
```

```
import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

```
# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create a version using "Save &
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session
```

 /kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/ryan-to-musk.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/trump-to-musk.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/musk-to-ryan.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/trump-to-Biden.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/ryan-to-taylor.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/taylor-to-margot.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/taylor-to-linus.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/biden-to-margot.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/taylor-to-ryan.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/ryan-to-biden.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/ryan-to-obama.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/taylor-to-obama.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/margot-to-obama.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/ryan-to-margot.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/margot-to-trump.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/trump-to-taylor.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/margot-to-taylor.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/musk-to-margot.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/biden-to-linus.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/linus-to-ryan.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/biden-to-Trump.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/biden-to-ryan.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/linus-to-margot.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/biden-to-musk.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/obama-to-margot.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/biden-to-Obama.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/trump-to-margot.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/linus-to-biden.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/obama-to-taylor.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/linus-to-trump.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/biden-to-taylor.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/margot-to-ryan.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/trump-to-linus.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/Obama-to-Biden.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/linus-to-taylor.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/trump-to-ryan.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/taylor-to-musk.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/margot-to-biden.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/FAKE/musk-to-linus.wav

```

/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/REAL/trump-original.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/REAL/taylor-original.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/REAL/obama-original.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/REAL/margot-original.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/REAL/linus-original.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/REAL/musk-original.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/REAL/ryan-original.wav
/kaggle/input/deep-voice-deepfake-voice-recognition/DEMONSTRATION/DEMONSTRATION/linus-original-DEMO.mp3
/kaggle/input/deep-voice-deepfake-voice-recognition/DEMONSTRATION/DEMONSTRATION/linus-to-musk-DEMO.mp3

```

Import Libraries

```
!pip install resampy
```

```

Collecting resampy
  Downloading resampy-0.4.3-py3-none-any.whl.metadata (3.0 kB)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.11/dist-packages (from resampy) (2.0.2)
Requirement already satisfied: numba>=0.53 in /usr/local/lib/python3.11/dist-packages (from resampy) (0.60.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.11/dist-packages (from numba>=0.53->resampy) (0.43.0)
  Downloading resampy-0.4.3-py3-none-any.whl (3.1 MB)
    3.1/3.1 MB 41.0 MB/s eta 0:00:00
Installing collected packages: resampy
Successfully installed resampy-0.4.3

```

```

import numpy as np
import pandas as pd
import os
import librosa
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
import IPython
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from imblearn.over_sampling import RandomOverSampler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation, Dropout, Conv2D, MaxPool2D, Flatten
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping

```

Load Data

```
audio_files_path = "/kaggle/input/deep-voice-deepfake-voice-recognition/KAGGLE/AUDIO/"
```

```

folders = os.listdir(audio_files_path)
print(folders)

```

```
['FAKE', 'REAL']
```

```

real_audio = "/kaggle/input/deep-voice-deepfake-voice-recognition/DEMONSTRATION/DEMONSTRATION/linus-original-DEMO.mp3"
fake_audio = "/kaggle/input/deep-voice-deepfake-voice-recognition/DEMONSTRATION/DEMONSTRATION/linus-to-musk-DEMO.mp3"

```

Visualization

```

print("Real Audio:")
#IPython.display.Audio(real_audio)

```

```
Real Audio:
```

```

print("Fake Audio:")
#IPython.display.Audio(fake_audio)

```

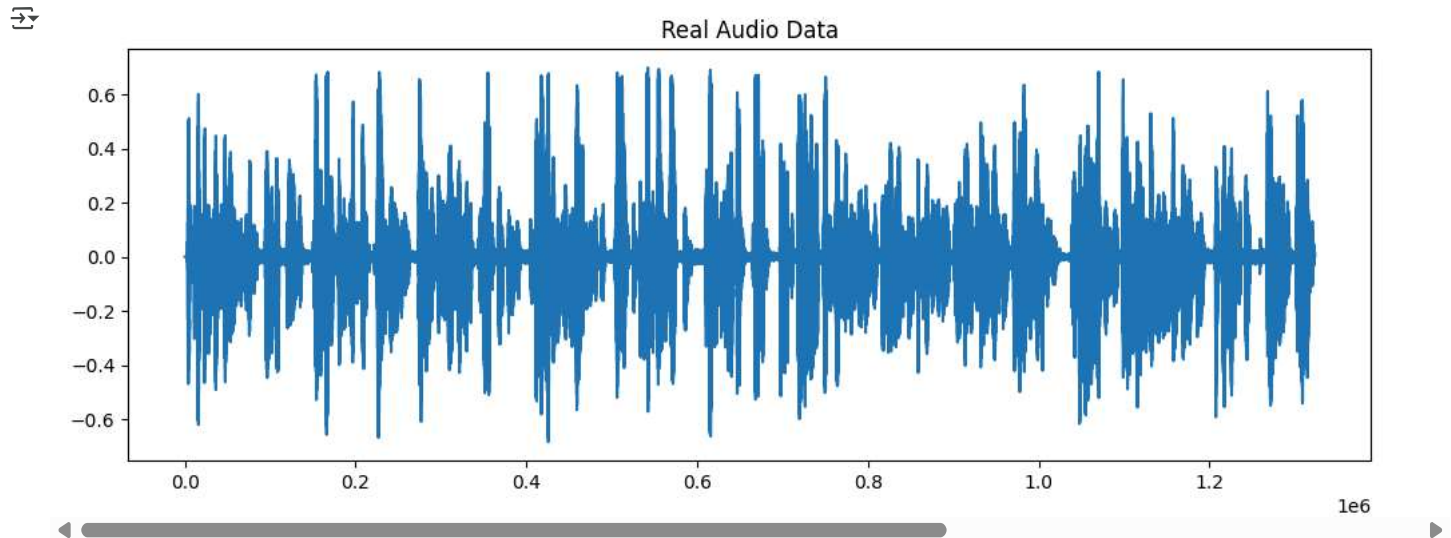
```
Fake Audio:
```

```

real_ad, real_sr = librosa.load(real_audio)
plt.figure(figsize=(12, 4))

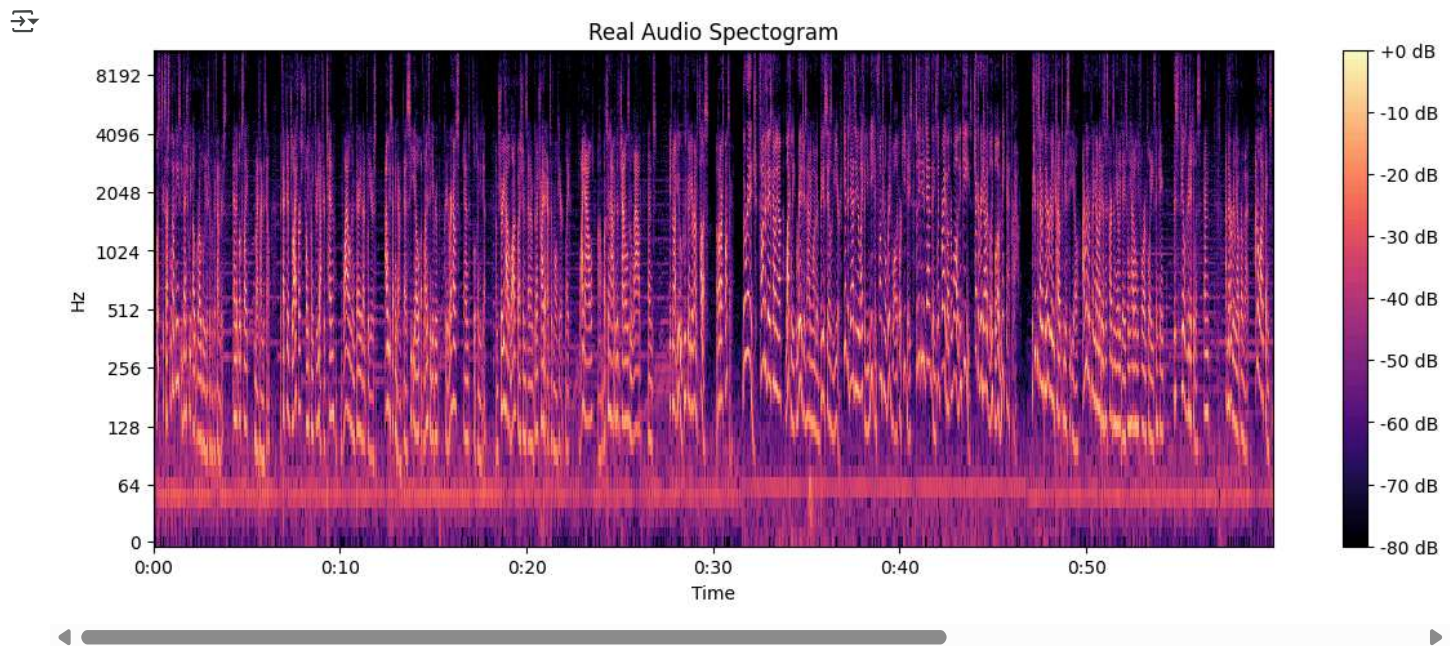
```

```
plt.plot(real_ad)
plt.title("Real Audio Data")
plt.show()
```



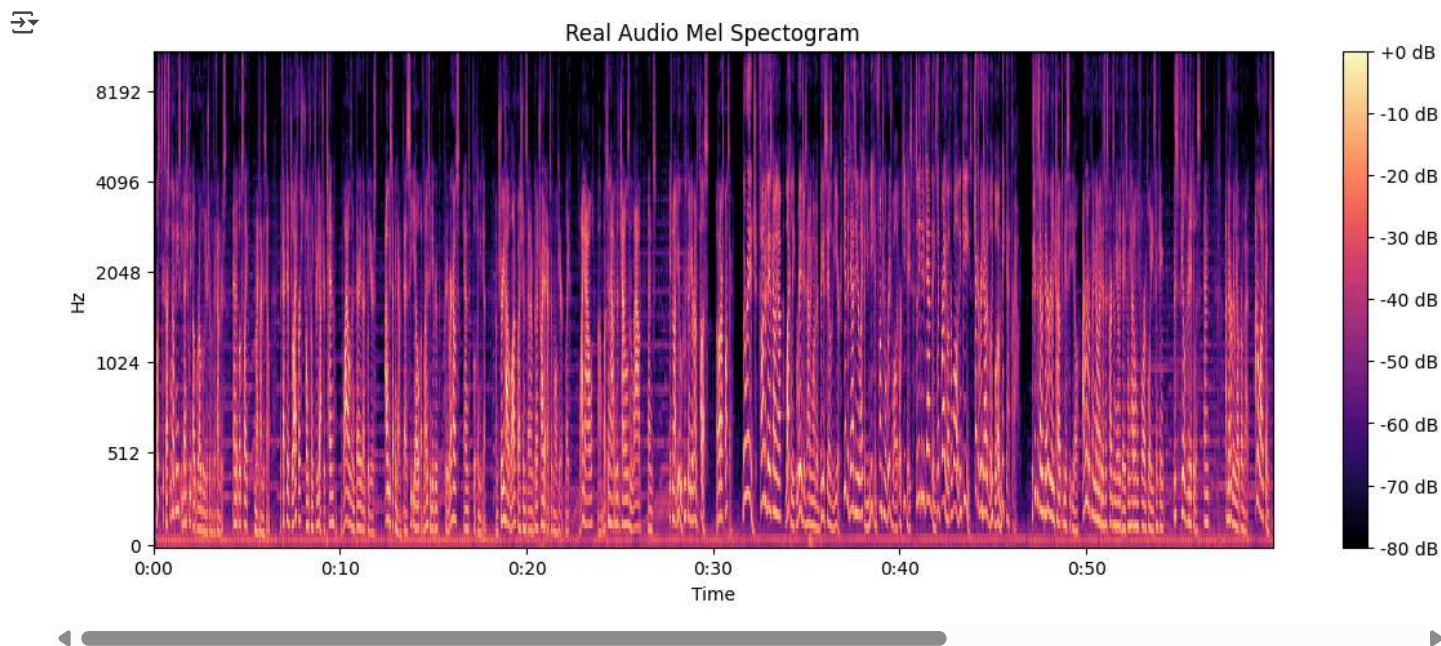
```
real_spec = np.abs(librosa.stft(real_ad))
real_spec = librosa.amplitude_to_db(real_spec, ref=np.max)

plt.figure(figsize=(14, 5))
librosa.display.specshow(real_spec, sr=real_sr, x_axis="time", y_axis="log")
plt.colorbar(format="%+2.0f dB")
plt.title("Real Audio Spectrogram")
plt.show()
```



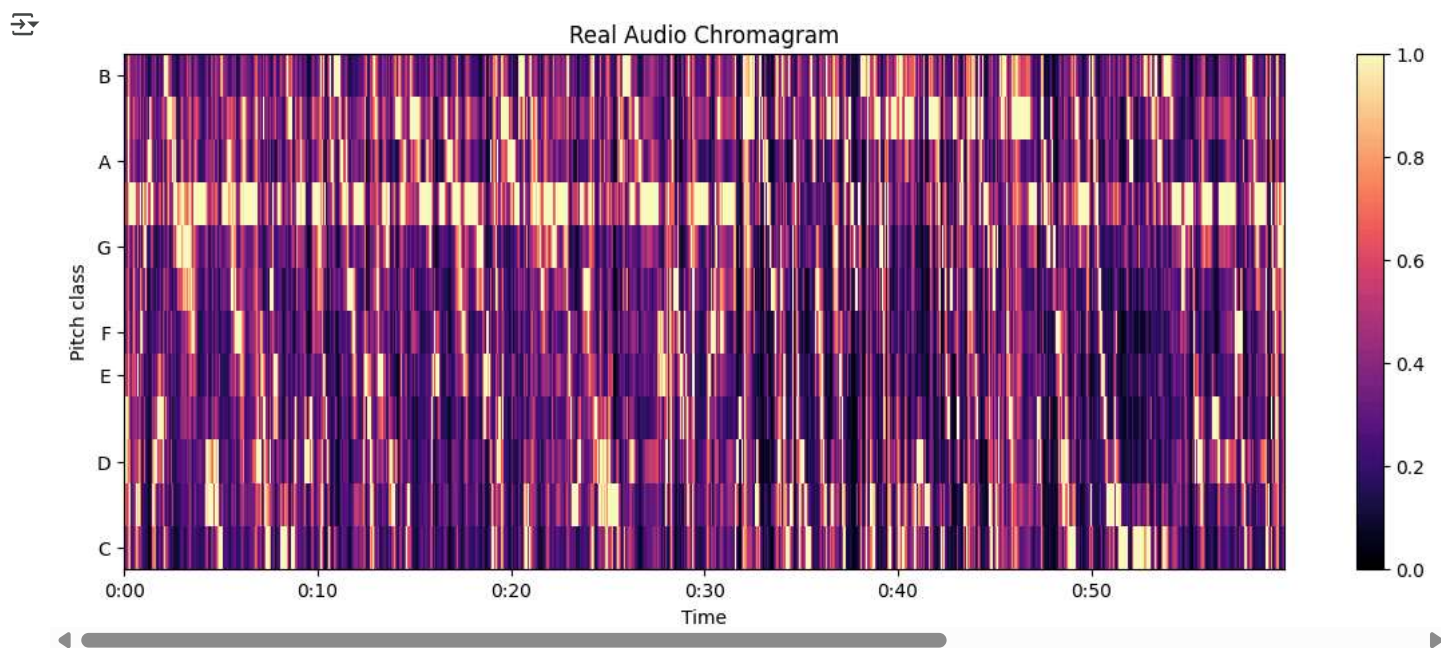
```
real_mel_spec = librosa.feature.melspectrogram(y=real_ad, sr=real_sr)
real_mel_spec = librosa.power_to_db(real_mel_spec, ref=np.max)

plt.figure(figsize=(14, 5))
librosa.display.specshow(real_mel_spec, y_axis="mel", x_axis="time")
plt.title("Real Audio Mel Spectrogram")
plt.colorbar(format="%+2.0f dB")
plt.show()
```



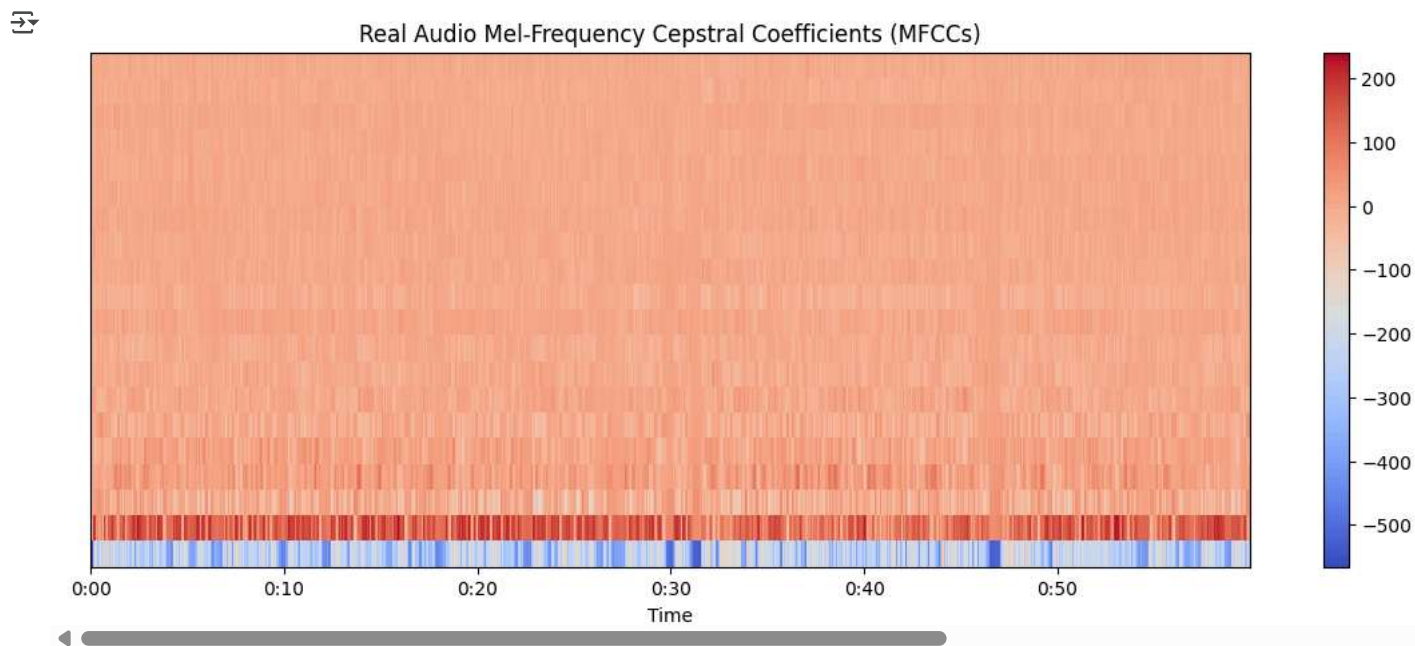
```
real_chroma = librosa.feature.chroma_cqt(y=real_ad, sr=real_sr, bins_per_octave=36)

plt.figure(figsize=(14, 5))
librosa.display.specshow(real_chroma, sr=real_sr, x_axis="time", y_axis="chroma", vmin=0, vmax=1)
plt.colorbar()
plt.title("Real Audio Chromagram")
plt.show()
```

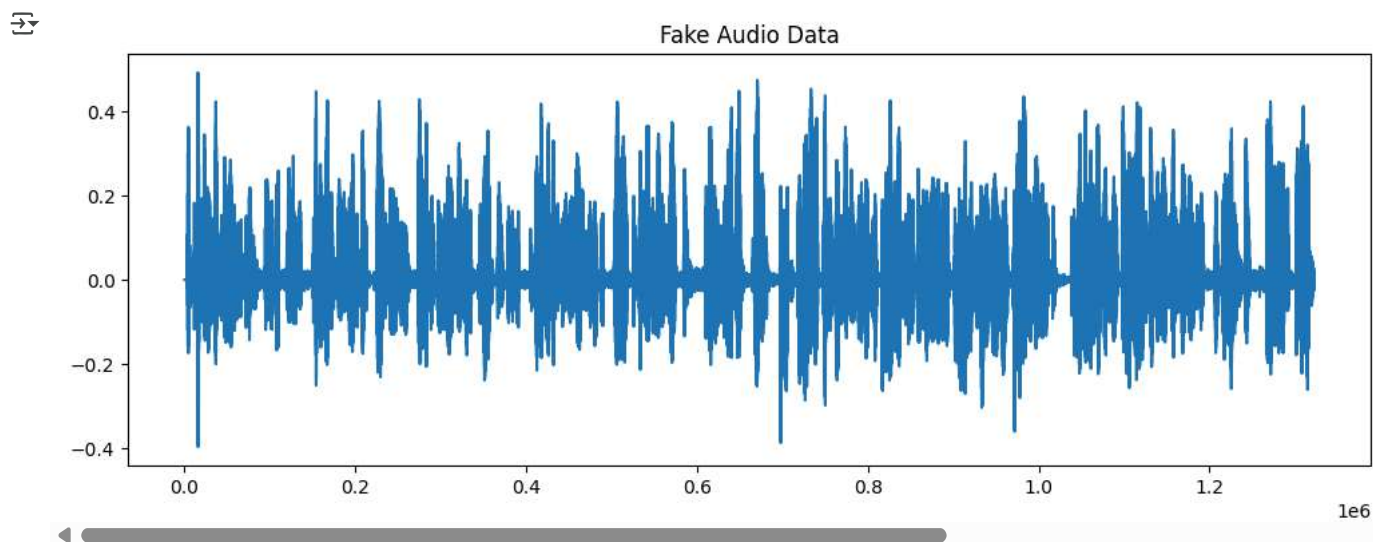


```
real_mfccs = librosa.feature.mfcc(y=real_ad, sr=real_sr)

plt.figure(figsize=(14, 5))
librosa.display.specshow(real_mfccs, sr=real_sr, x_axis="time")
plt.colorbar()
plt.title("Real Audio Mel-Frequency Cepstral Coefficients (MFCCs)")
plt.show()
```

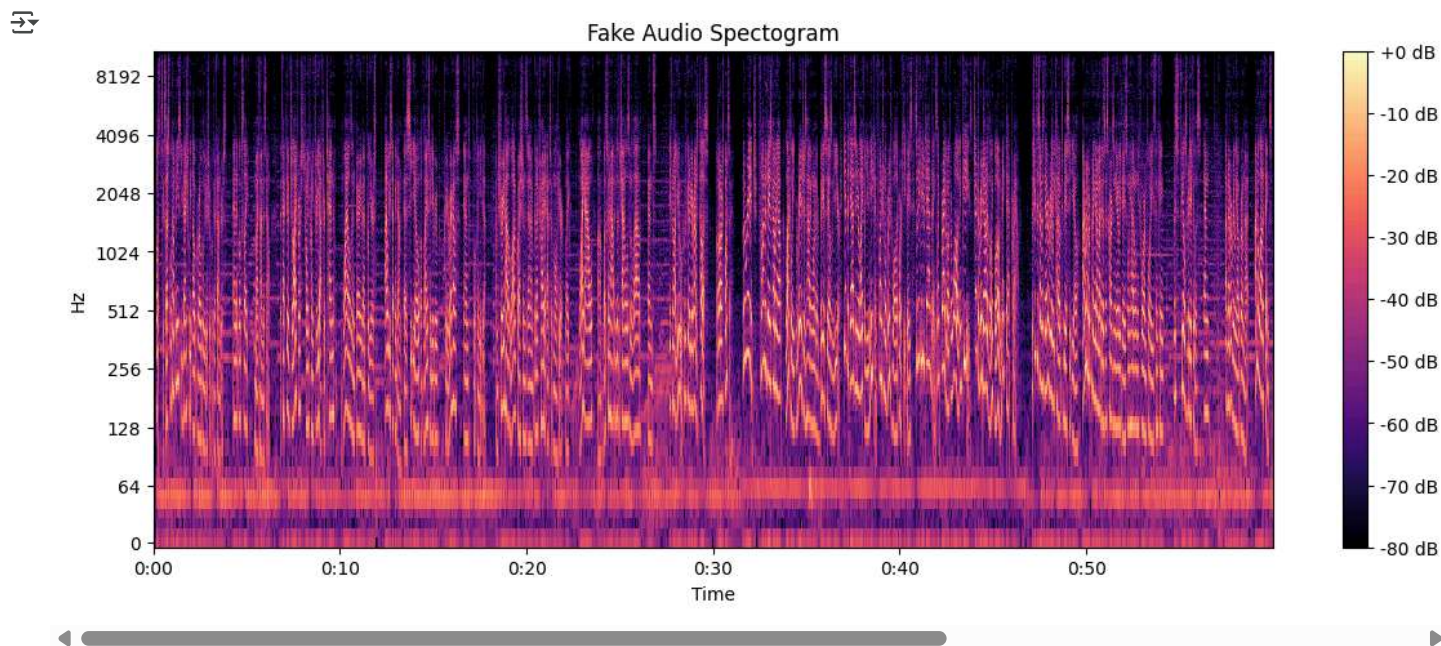


```
fake_ad, fake_sr = librosa.load(fake_audio)
plt.figure(figsize=(12, 4))
plt.plot(fake_ad)
plt.title("Fake Audio Data")
plt.show()
```



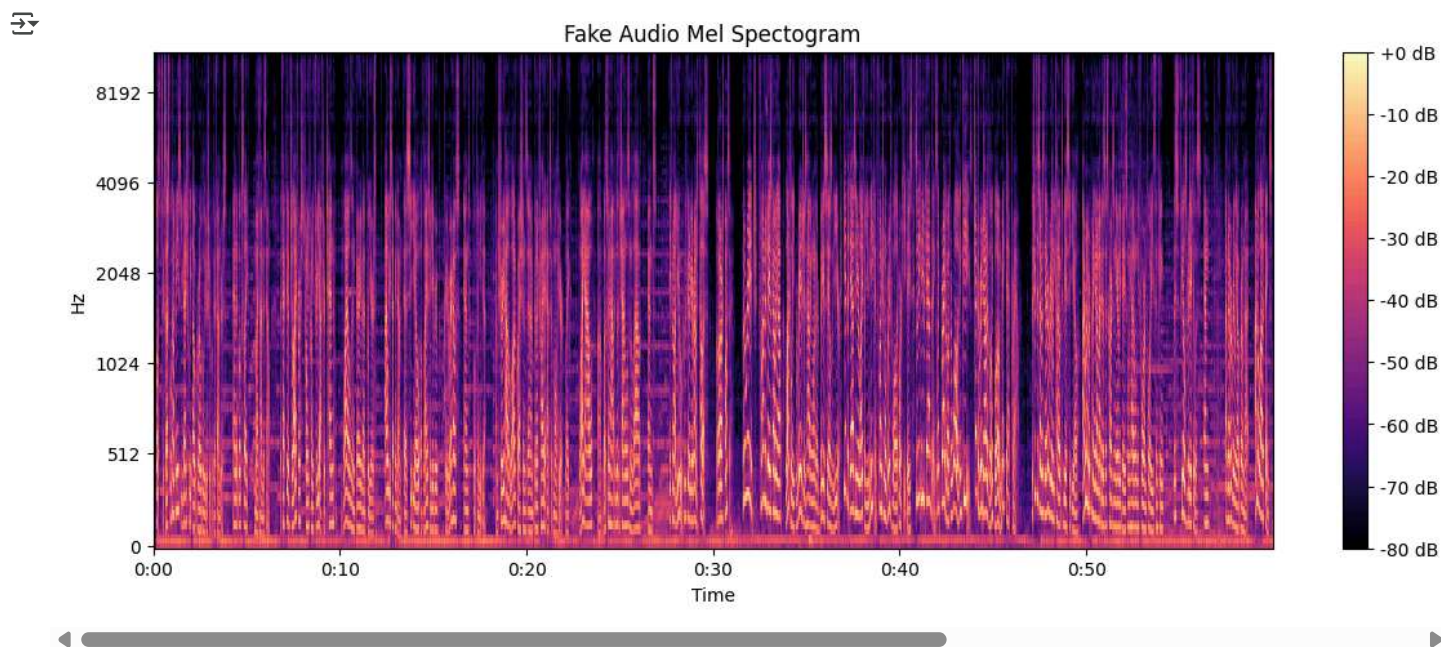
```
fake_spec = np.abs(librosa.stft(fake_ad))
fake_spec = librosa.amplitude_to_db(fake_spec, ref=np.max)

plt.figure(figsize=(14, 5))
librosa.display.specshow(fake_spec, sr=fake_sr, x_axis="time", y_axis="log")
plt.colorbar(format="%+2.0f dB")
plt.title("Fake Audio Spectrogram")
plt.show()
```



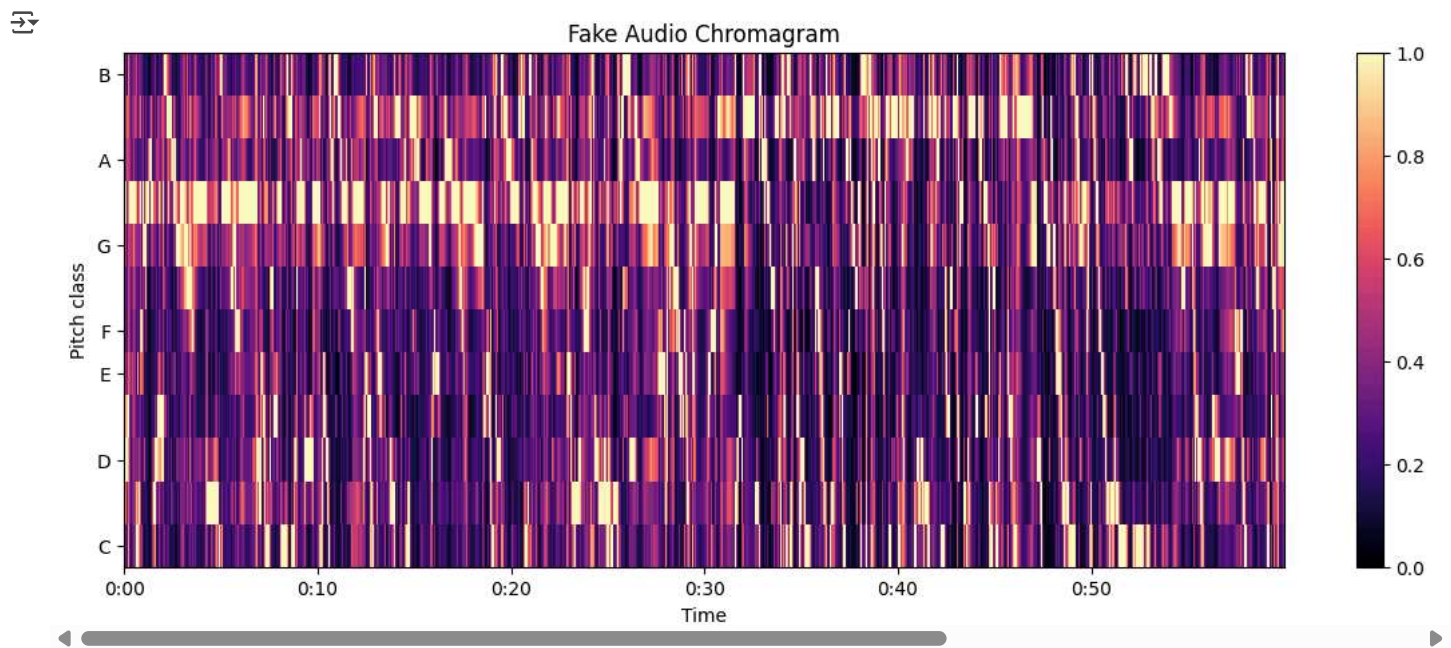
```
fake_mel_spect = librosa.feature.melspectrogram(y=fake_ad, sr=fake_sr)
fake_mel_spect = librosa.power_to_db(fake_mel_spect, ref=np.max)
```

```
plt.figure(figsize=(14, 5))
librosa.display.specshow(fake_mel_spect, y_axis="mel", x_axis="time")
plt.title("Fake Audio Mel Spectrogram")
plt.colorbar(format="%+2.0f dB")
plt.show()
```



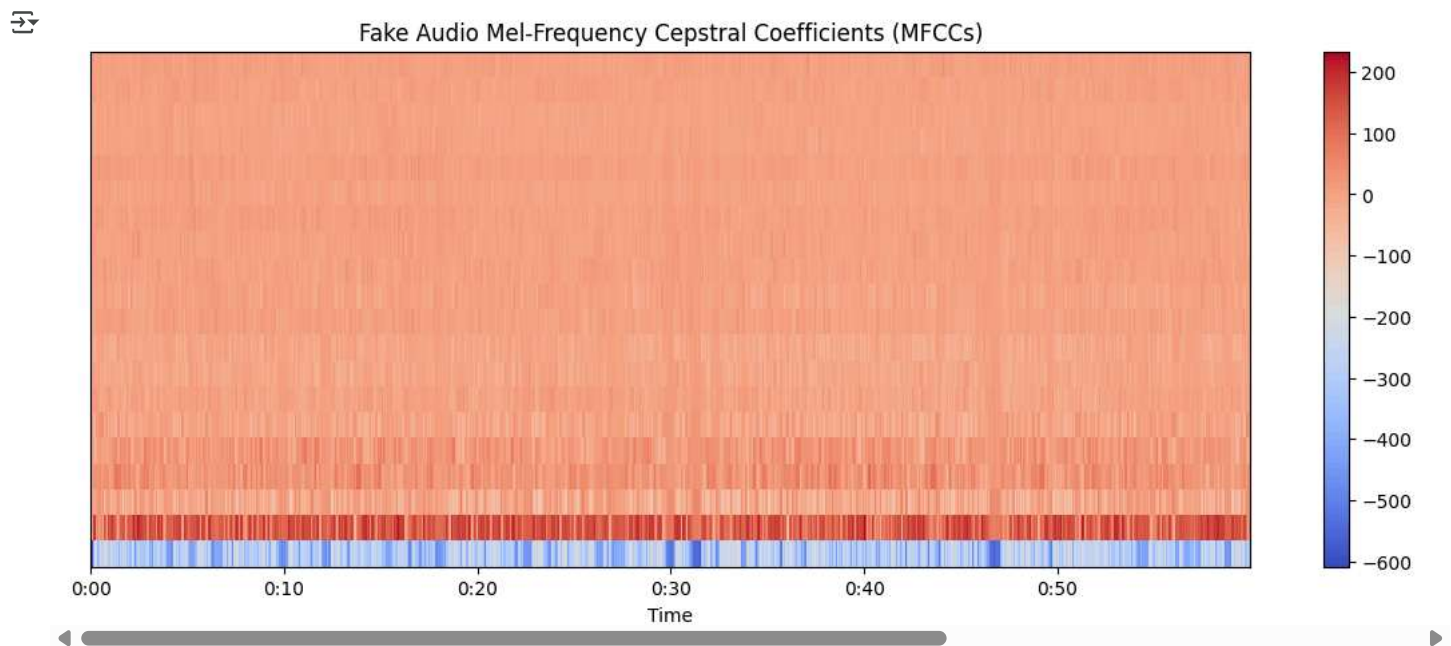
```
fake_chroma = librosa.feature.chroma_cqt(y=fake_ad, sr=fake_sr, bins_per_octave=36)
```

```
plt.figure(figsize=(14, 5))
librosa.display.specshow(fake_chroma, sr=fake_sr, x_axis="time", y_axis="chroma", vmin=0, vmax=1)
plt.colorbar()
plt.title("Fake Audio Chromagram")
plt.show()
```



```
fake_mfccs = librosa.feature.mfcc(y=fake_ad, sr=fake_sr)

plt.figure(figsize=(14, 5))
librosa.display.specshow(fake_mfccs, sr=fake_sr, x_axis="time")
plt.colorbar()
plt.title("Fake Audio Mel-Frequency Cepstral Coefficients (MFCCs)")
plt.show()
```



✓ Preprocess

```
data = []
labels = []

for folder in folders:
    files = os.listdir(os.path.join(audio_files_path, folder))
    for file in tqdm(files):
        file_path = os.path.join(audio_files_path, folder, file)
        audio, sample_rate = librosa.load(file_path, res_type="kaiser_fast")
        mfccs_features = librosa.feature.mfcc(y=audio, sr=sample_rate, n_mfcc=40)
        mfccs_features_scaled = np.mean(mfccs_features.T, axis=0)
        data.append(mfccs_features_scaled)
        labels.append(folder)
```

```
100% |██████████| 56/56 [06:29<00:00, 6.95s/it]
100% |██████████| 8/8 [00:58<00:00, 7.36s/it]
```

```
feature_df = pd.DataFrame({"features": data, "class": labels})
feature_df.head()
```

```
features  class
0  [-291.10123, 95.36503, -13.482852, 10.591423, ...] FAKE
1  [-250.76636, 117.728546, -22.377504, 30.396112...] FAKE
2  [-256.0186, 108.66988, 2.0284197, 24.126278, 2...] FAKE
3  [-295.43436, 76.94639, -0.6353918, 17.214228, ...] FAKE
4  [-288.0438, 75.1327, 1.3050734, 20.612984, -2....] FAKE
```

Next steps: [Generate code with feature_df](#) [View recommended plots](#) [New interactive sheet](#)

```
feature_df["class"].value_counts()
```

```
count
class
FAKE    56
REAL     8
```

```
def label_encoder(column):
    le = LabelEncoder().fit(column)
    print(column.name, le.classes_)
    return le.transform(column)
```

```
feature_df["class"] = label_encoder(feature_df["class"])
```

```
class ['FAKE' 'REAL']
```

Feature Scaling

```
X = np.array(feature_df["features"].tolist())
y = np.array(feature_df["class"].tolist())
```

```
ros = RandomOverSampler(random_state=42)
X_resampled, y_resampled = ros.fit_resample(X, y)
```

```
y_resampled = to_categorical(y_resampled)
```

```
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled, test_size=0.2, random_state=42)
```

```
num_labels = len(feature_df["class"].unique())
num_labels
```

```
2
```

```
input_shape = feature_df["features"][0].shape
input_shape
```

```
(40,)
```

Model

```

model = Sequential()
model.add(Dense(128, input_shape=input_shape))
model.add(Activation(activation="relu"))
model.add(Dropout(0.5))
model.add(Dense(256))
model.add(Activation(activation="relu"))
model.add(Dropout(0.5))
model.add(Dense(128))
model.add(Activation(activation="relu"))
model.add(Dropout(0.5))
model.add(Dense(num_labels))
model.add(Activation(activation="softmax"))

```

→ /usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to `super().__init__(activity_regularizer=activity_regularizer, **kwargs)`

```
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
```

```
model.summary()
```

→ Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	5,248
activation (Activation)	(None, 128)	0
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 256)	33,024
activation_1 (Activation)	(None, 256)	0
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 128)	32,896
activation_2 (Activation)	(None, 128)	0
dropout_2 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 2)	258
activation_3 (Activation)	(None, 2)	0

Total params: 71.426 (279.01 KB)

```
early = EarlyStopping(monitor="val_loss", patience=5)
```

```
history = model.fit(X_train, y_train, validation_data=(X_test, y_test), batch_size=2, epochs=100, callbacks=[early])
```

→ Epoch 1/100
45/45 ————— 7s 67ms/step - accuracy: 0.4897 - loss: 21.9072 - val_accuracy: 0.3478 - val_loss: 12.3237
Epoch 2/100
45/45 ————— 0s 8ms/step - accuracy: 0.5181 - loss: 22.7474 - val_accuracy: 0.3478 - val_loss: 16.2372
Epoch 3/100
45/45 ————— 0s 5ms/step - accuracy: 0.4210 - loss: 24.1560 - val_accuracy: 0.6087 - val_loss: 1.1450
Epoch 4/100
45/45 ————— 0s 4ms/step - accuracy: 0.5347 - loss: 13.1047 - val_accuracy: 0.6522 - val_loss: 2.5587
Epoch 5/100
45/45 ————— 0s 4ms/step - accuracy: 0.5465 - loss: 9.1666 - val_accuracy: 0.6522 - val_loss: 1.5660
Epoch 6/100
45/45 ————— 0s 4ms/step - accuracy: 0.5331 - loss: 7.5882 - val_accuracy: 0.6522 - val_loss: 2.2083
Epoch 7/100
45/45 ————— 0s 4ms/step - accuracy: 0.5281 - loss: 7.1652 - val_accuracy: 0.6522 - val_loss: 2.3458
Epoch 8/100
45/45 ————— 0s 4ms/step - accuracy: 0.5885 - loss: 4.4772 - val_accuracy: 0.6522 - val_loss: 1.3241

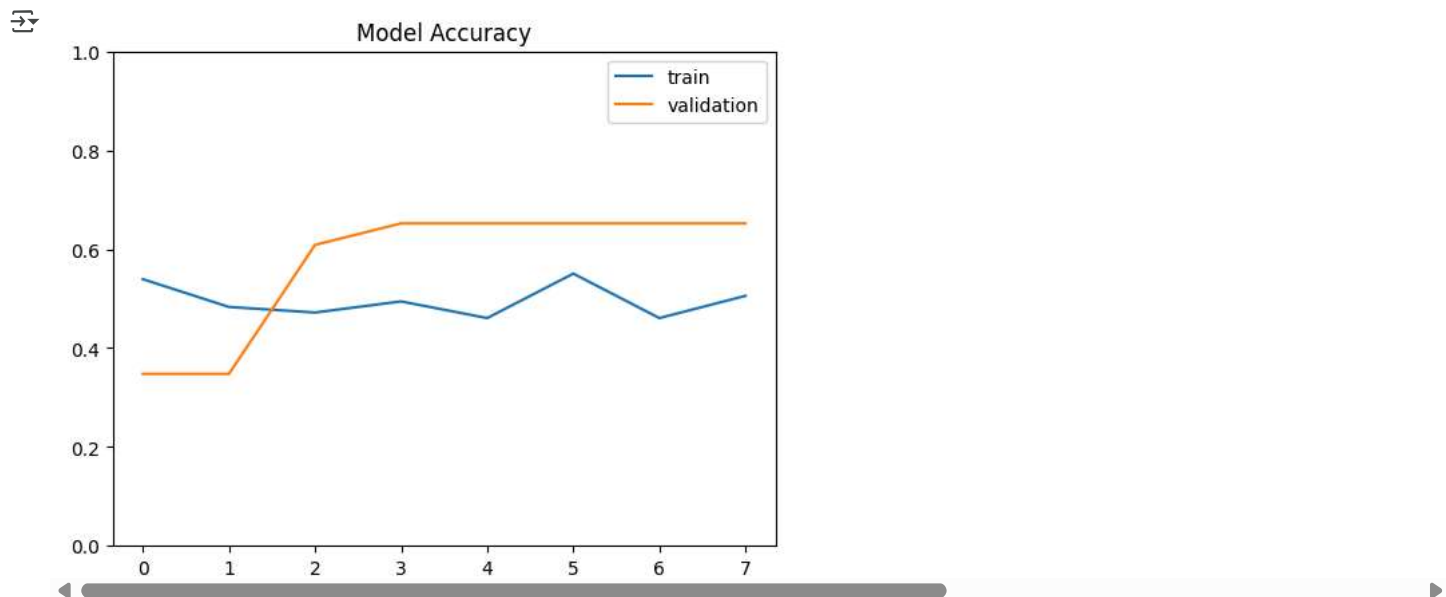
```

test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Loss:", test_loss)
print("Test Accuracy:", test_acc)

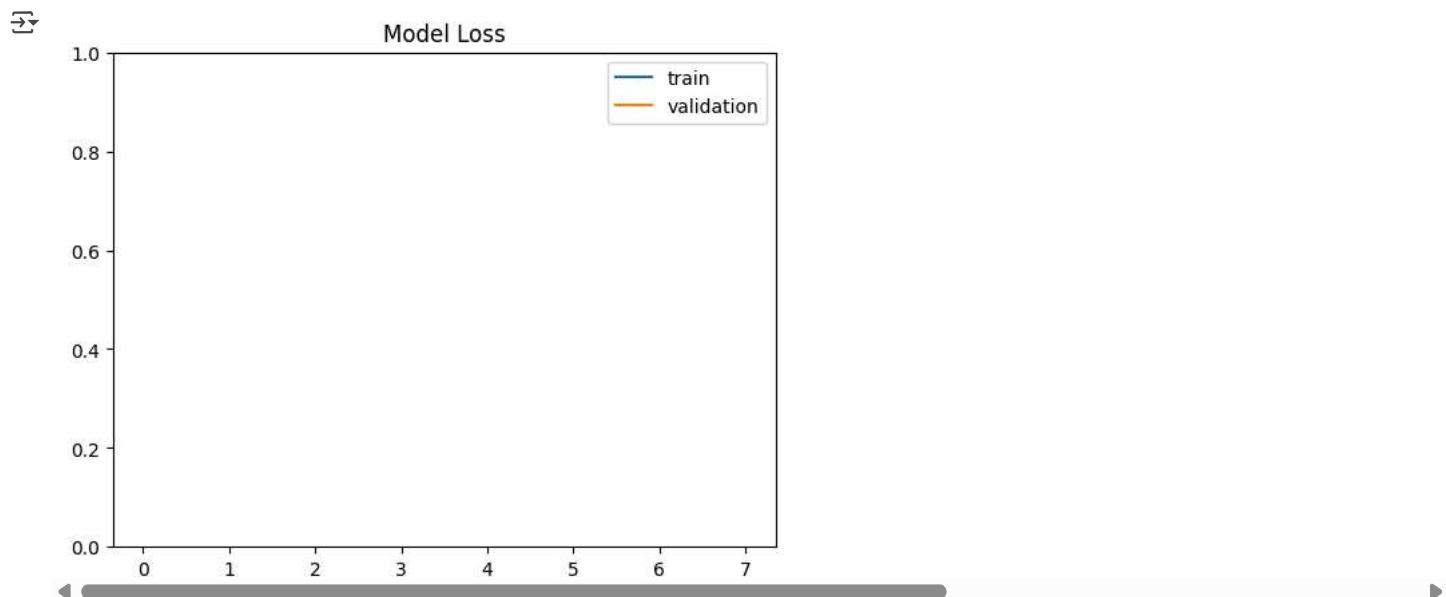
```

→ 1/1 ————— 0s 400ms/step - accuracy: 0.6522 - loss: 1.3241
Test Loss: 1.324137568473816
Test Accuracy: 0.6521739363670349

```
plt.figure()
plt.title("Model Accuracy")
plt.plot(history.history["accuracy"], label="train")
plt.plot(history.history["val_accuracy"], label="validation")
plt.legend()
plt.ylim([0, 1])
plt.show()
```



```
plt.figure()
plt.title("Model Loss")
plt.plot(history.history["loss"], label="train")
plt.plot(history.history["val_loss"], label="validation")
plt.legend()
plt.ylim([0, 1])
plt.show()
```



▼ Test

```
def detect_fake(filename):
    sound_signal, sample_rate = librosa.load(filename, res_type="kaiser_fast")
    mfcc_features = librosa.feature.mfcc(y=sound_signal, sr=sample_rate, n_mfcc=40)
    mfccs_features_scaled = np.mean(mfcc_features.T, axis=0)
    mfccs_features_scaled = mfccs_features_scaled.reshape(1, -1)
    result_array = model.predict(mfccs_features_scaled)
    print(result_array)
    result_classes = ["FAKE", "REAL"]
```

```
result = np.argmax(result_array[0])  
print("Result:", result_classes[result])
```

```
test_real = "/kaggle/input/deep-voice-deepfake-voice-recognition/DEMONSTRATTON/DEMONSTRATTON/linus-original-DEMO.mp3"
```